ADR-005: Phase A 運営方針 — CLI駆動・MCP禁止

ステータス

承認

日付

2026-03-12

コンテキスト

シャチョツーを**自社（開発チーム）の日常業務オペレーション**にまず適用し、実際の業務価値を検証してからプロダクト化するアプローチを取る。これを **Phase A**（内部適用フェーズ）と呼ぶ。

Phase A の目的

1. **自社業務でのPMF検証**: 開発・営業・経理・採用等の日常業務をAIエージェントで自動化し、「社長の代理人」ユースケースの有効性を実証する
2. **ビルド対象の特定**: 実際に使ってみることで「次に何を作るべきか」の優先度を実地で発見する（EatYourOwnDogFood）
3. **オペレーションコスト削減**: Phase B（プロダクト公開）前から自社の管理工数を削減し、開発リソースをコードに集中させる

Phase A の対象業務

業務	使用するCLIツール	自動化内容
コード管理・レビュー	`claude`, `gh`	PR作成・コードレビュー・Issue管理
コミュニケーション	`slack-cli`	定型連絡・進捗報告・アラート
経理・請求	`freee-cli`	請求書作成・経費仕訳・月次集計
インフラ管理	`gcloud`	デプロイ・ログ確認・スケール調整
ドキュメント管理	`notion-cli`, `claude`	ドキュメント生成・更新・構造化
採用	`claude`, `gh`	JD作成・応募者スクリーニング補助

選択肢の検討背景

AI エージェントと外部SaaS連携を実現する技術手段として、大きく以下が存在する:

1. **CLI（コマンドラインインターフェース）**：各SaaSが提供するCLIツール（`gh`、`gcloud`、`slack-cli`等）をシェルスクリプトやPythonから呼び出す
2. **MCP（Model Context Protocol）**：AnthropicのMCPサーバーを経由して、LLMがSaaS APIを自律的に呼び出す
3. **iPaaS（n8n, Zapier等）**：ノーコードのワークフロー自動化プラットフォーム
4. **直接API呼び出し（Python requests/httpx）**：各SaaSのREST APIを直接呼び出すPythonコードを書く

Phase A ではこのいずれを使うかを判断する必要があった。

検討した選択肢

選択肢A: CLI駆動（`claude`, `gh`, `gcloud`, `slack-cli` 等）— 採用案

概要: 各SaaSが公式提供するCLIツールを `subprocess` またはシェルスクリプトから呼び出す。AIによる判断は `claude` CLIを通じてインタラクティブに、またはヘッドレスモードで実行する。

メリット:

1. **自然な監査ログ**: `bash_history`・`script` コマンド・ターミナルセッションログが自動的に監査記録になる。「誰がいつ何のコマンドを実行したか」が明確。エンタープライズのコンプライアンス要件（将来の顧客向け）への対応も容易

`bash # この1行が完全な監査ログになる 2026-03-12T09:23:11 gh pr create --title "feat: ナレッジ抽出API" --body "..."`

2. **再現性・再実行可能性**: 失敗したステップを同じコマンドで再実行できる。デバッグ時に「どのステップで失敗したか」を特定してそこから再開できる
3. **スクリプト化・自動化**: CLIコマンドをシェルスクリプトや `Makefile` にまとめれば、定型業務の完全自動化が容易

`bash # scripts/weekly_report.sh #!/bin/bash claude "先週のGitHub活動を要約" | slack-cli send --channel #management freee-cli invoice list --period this-month | claude "請求状況を要約"`

4. **透明性・デバッグ容易性**: 各ステップの入出力が標準入出力として見える。MCPのような「中間抽象層のブラックボックス」がない。問題発生時の原因特定が速い
5. **移植性**: 同じスクリプトがどの環境でも動く（Docker・CI・ローカル・Cloud Run Jobs）。MCPはサーバー設定が環境依存になる
6. **学習効果**: 実際に使うCLIコマンドを積み重ねることで「何の操作がどの程度の頻度で必要か」が可視化される。これがプロダクト（Phase B）のBPO Worker設計の仕様になる

> 「CLIで100回やった操作が、プロダクトの自動化対象になる」

デメリット:
- **冗長性**: MCPと比較してより多くのコードを書く必要がある場合がある
- **並列化の難しさ**: 複数のCLI操作を並列で実行するには、明示的な `&` やxargs等が必要
- **CLIバージョン管理**: 各ツールのCLIバージョンが変わるとスクリプトが壊れることがある
- 緩和策: `requirements.txt` 相当のCLIバージョンを `scripts/versions.txt` で管理

選択肢B: MCP（Model Context Protocol）— 禁止

概要: AnthropicのMCPプロトコルを使い、LLMが直接SaaS APIを呼び出す。`claude` CLIにMCPサーバーを登録することで、自然言語でSaaS操作が可能になる。

メリット:
- 自然言語で高度なSaaS操作が可能（「先月の売上をfreeeから取ってきてSlackで報告して」）
- 手続きコードを書かなくてよい
- LLMが文脈を理解して柔軟に対応できる

デメリット:

1. **デバッグ困難**: MCPサーバーを経由するとLLMが「なぜその操作をしたか」の過程が見えにくい。特に失敗時の原因特定が難しい
2. **監査ログの不完全性**: MCP操作はLLMの内部状態に依存するため、「同じプロンプトでも毎回同じ操作をするとは限らない」。監査要件（いつ・誰が・何をしたか）を満たすログが取りにくい

3. **再現性なし**: LLMの出力は確率的。同じ入力でも異なる操作になることがある。重要な業務（請求書発行等）での使用はリスクが高い
4. **透明性の欠如**: MCPは抽象化レイヤーが厚く、「どのAPIエンドポイントが呼ばれたか」「何のパラメータが送られたか」が見えにくい。セキュリティ監査が困難
5. **学習効果の欠如**: MCPに頼ると「実際に何の操作が必要か」が学習されない。Phase B でBPO Workerを設計する際に「何を自動化すべきか」の知識が蓄積されない
6. **MCPサーバー管理コスト**: 各SaaS向けのMCPサーバーを設定・維持・バージョン管理する必要がある。Phase A では余分な管理コスト
7. **セキュリティリスク**: MCPサーバーに各SaaSのAPIキーを渡すことになり、鍵の管理・漏洩リスクが増える。CLIツールはローカル設定ファイル（`~/.config/gh/` 等）で鍵を管理するためより安全

→ **決定**: Phase A では MCP を明示的に禁止する。Phase B（プロダクト開発）で「BPO Workerの内部実装」として採用を検討するが、社内業務運用での使用は禁止。

選択肢C: n8n / Zapier (iPaaS) — 却下

概要: ノーコードのワークフロー自動化プラットフォーム。GUIでフローを設計し、SaaS間の連携を実現する。

メリット:
- GUIで直感的にフローを設計できる
- 数百のSaaS連携が標準搭載
- トリガーベースの自動化が容易

デメリット:

1. **コードオーナーシップなし**: フロー定義がn8n/Zapierのプラットフォーム上に存在し、Gitで管理できない。バージョン管理・差分確認・レビューができない
2. **LLM統合の制限**: カスタムLLMロジック（シャチョツーのブレイン）との統合が困難。HTTP Requestノードでの呼び出しは可能だが、状態管理・条件分岐の柔軟性が低い
3. **戦略的矛盾**: シャチョツー自体がBPO Workerでこのカテゴリの代替製品を作る。競合ツールを内部で使うことは「自社プロダクトで解決できない問題がある」という自己矛盾を生む
4. **コスト**: Zapierの本番プランは月数万円。初期フェーズのコスト最小化に反する
5. **移植性ゼロ**: Phase B でBPO Workerを実装する際、n8nフローをPython/LangGraphに移植する工数が発生

→ **決定**: 戦略的矛盾とコードオーナーシップの問題から却下。

選択肢D: 直接API呼び出し（Python）— Phase B 向け

概要: 各SaaSのREST APIをPython（`httpx`等）で直接呼び出す。

メリット:
- 完全なコード制御
- テスト可能
- LangGraph等との統合が容易

デメリット:
- **初期セットアップ**: OAuth認証フロー・トークンリフレッシュ・エラーハンドリングを全て実装する必要がある
- **Phase A には過大**: 社内業務自動化の検証フェーズに本格的なAPIクライアント実装は時間がかかりすぎる

→ **決定**: Phase B（BPO Worker 実装）での採用が適切。Phase A では CLI を使ってユースケースを検証してから、必要なAPIを特定して実装する。

決定

Phase A（内部適用フェーズ）において、全てのエージェント操作をCLIコマンド経由で実施する。MCPの使用を禁止する。

具体的なルール:

```
許可: claude CLI, gh CLI, gcloud CLI, slack-cli, freee-cli, その他公式CLIツール
禁止: MCP サーバー経由の操作
禁止: n8n, Zapier 等のiPaaSプラットフォーム
判断保留: 直接API呼び出し（Phase B実装時に評価）
```

実装パターン

パターン1: シェルスクリプトによる定型業務自動化

```
#!/bin/bash
# scripts/daily_standup.sh
# 毎朝9:00にCron実行

set -euo pipefail

# 昨日のPR・Issueを要約
GITHUB_SUMMARY=$(gh pr list --state merged --limit 10 | \
  claude "これらのPRを1段落で要約してください")

# Slackに投稿
slack-cli message send \
  --channel "#standup" \
  --text "📊 昨日の開発進捗: ${GITHUB_SUMMARY}"
```

パターン2: Pythonによる複合ワークフロー

```
# scripts/monthly_invoice.py
import subprocess
import json

def run_cli(cmd: list[str]) -> str:
    """CLIコマンドを実行して標準出力を返す"""
    result = subprocess.run(cmd, capture_output=True, text=True, check=True)
    return result.stdout

# freeeから今月の取引を取得
invoices_json = run_cli(["freee-cli", "invoice", "list", "--period", "this-month", "--
format", "json"])
invoices = json.loads(invoices_json)

# claudeで支払い状況をサマリー
summary = run_cli(["claude", "-p", f"以下の請求書リストから未払いを抽出してください:
{invoices_json}"])

# Slackに通知
run_cli(["slack-cli", "message", "send", "--channel", "#finance", "--text", summary])
```

パターン3: Makefileによるタスク管理

```
# Makefile
.PHONY: deploy report review

deploy:
    gcloud run deploy shachotwo-api --image gcr.io/$(PROJECT)/api:latest
    gcloud run deploy shachotwo-frontend --image gcr.io/$(PROJECT)/frontend:latest

weekly-report:
    ./scripts/weekly_report.sh

review-prs:
    gh pr list --state open | claude "各PRのレビューポイントを指摘してください"
```

Phase B への引き継ぎ

Phase A で蓄積したCLIオペレーション履歴を分析し、以下を特定する:
1. 月20回以上実行されたCLIコマンド → BPO Workerの優先実装対象
2. エラーが多かったオペレーション → 品質向上の余地
3. 人間の判断が必要だったステップ → HitL設計のインプット

この分析が `workers/bpo/` の設計仕様書になる。

影響

ポジティブな影響

- **監査証跡の自動取得**: bash_historyと `script` コマンドによる全操作記録が自動的に監査ログになる。将来の顧客向けコンプライアンス対応の実証例として活用できる
- **BPO Workerの仕様収集**: 実際の業務で使ったCLIコマンドが、そのままBPO Workerが自動化すべきオペレーションのリストになる。座学ではなく実地で仕様が決まる
- **開発チームのCLI習熟度向上**: 全メンバーが各ツールのCLIを使いこなすことで、将来のBPO Worker実装時にAPIの仕様・制約を正確に把握できる
- **コスト最小化**: MCPサーバー設定・iPaaSサブスクリプション費用が不要

負の影響・トレードオフ

- **初期の冗長性**: MCPやiPaaSに比べて初期のスクリプト記述量が多い。特に認証フロー（初回セットアップ）に時間がかかる
- 緩和策: `scripts/setup.sh` で初期設定を一括実行するスクリプトを整備する
- **並列化の手動実装**: 複数CLIを並列実行するには明示的なシェルの並列化（`&` + `wait`）が必要。MCPのように自動では並列化されない

- **CLIの機能制限**: 一部のSaaSはCLIで提供されていない機能がある。その場合は直接API呼び出しを検討する（都度判断）

今後の制約

1. **MCP使用禁止（Phase A期間）**: Phase A 終了（≈ Phase B開始）まではMCPの使用を禁止する。CI/CDやコードレビューでMCPサーバー設定のコミットをブロックする
2. **CLIコマンドのドキュメント化**: 使用したCLIコマンドは `scripts/README.md` に記録し、再現性を保つ
3. **APIキーはCLIのデフォルト設定で管理**: `~/.config/` 配下の設定ファイルを使用し、スクリプト内にAPIキーをハードコードしない
4. **Phase A終了条件**: 以下を満たしたらPhase Aをクローズして Phase Bへ移行する
   - 主要3業務（コード管理・コミュニケーション・経理）の定型オペレーションを80%自動化
   - CLIオペレーションログ100件以上を蓄積してBPO Worker仕様を確定

関連

- `shachotwo/03_実装計画.md` — Phase A/B 定義
- `shachotwo/04_開発ステップ.md` — Step別ビジネスフェーズ
- `scripts/` — CLIスクリプト格納ディレクトリ
- ADR-003: LangGraph採用（Phase B でのBPO Worker実装で使用。Phase A ではCLIで代替）